

Лабораторная работа № 4

«Обтекание цилиндра потоком воды»

Аннотация

Схема течения вязкой жидкости при обтекании цилиндра существенно зависит от числа Рейнольдса, которое основано на диаметре цилиндра. При низких числах Рейнольдса ($4 < Re < 60$) на задней стороне цилиндра образуются два устойчивых вихря, которые остаются прикрепленными к цилиндру. При более высоких числах Рейнольдса поток становится неустойчивым, появляется дорожка Кармана.

Цель данной работы – это рассмотреть пример внешнего несжимаемого потока. Вода при температуре 293.2 К обтекает цилиндр диаметром 0.01 м.

Необходимо выполнить расчеты для числа Рейнольдса 41 и 140, после сравнить полученные результаты с результатами из SOLIDWORKS FLOW SIMULATION 2018 (validations examples) 2018 г. пример: Flow over a Circular Cylinder.

План занятия

- определение геометрии;
- вычисление скорости;
- настройка файлов решения;
- построение сетки;
- выполнение расчета для числа Рейнольдса 41;
- выполнение расчета для числа Рейнольдса 140;
- сравнение результатов.

Форма представления выполненных заданий

Полученные в рамках данной лабораторной работы результаты прикладываются в качестве ответа в том же разделе LMS.

1 ОПРЕДЕЛЕНИЕ ГЕОМЕТРИИ

Вся расчетная область разделена на 6 блоков (Рисунок 1). Точкой (0 0 0) будем считать центр цилиндра (на рисунке центр квадрата, который будет вписан в окружность).

Задача двумерная, но так как OpenFOAM рассматривает все сетки как трехмерные, то необходимо описать толщину всего в одну ячейку в третьем направлении, в котором расчет выполняться не будет. На рисунке точки задней плоскости геометрии показаны в скобках.

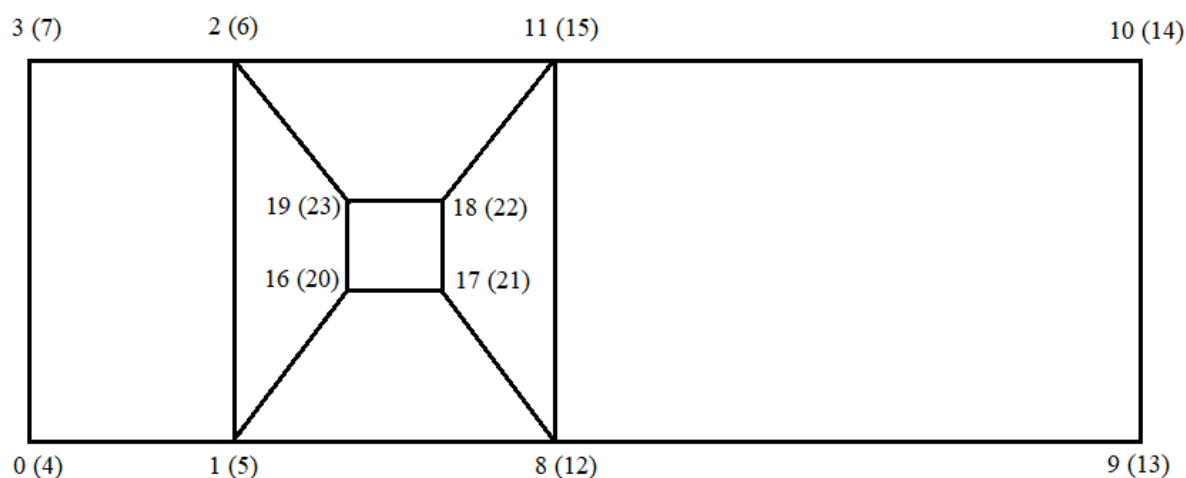


Рисунок 1 – Геометрия задачи

Для решения задачи будем использовать решатель `pimpleFoam`, который предназначен для несжимаемого потока. Он находится в `tutorials => incompressible => pimpleFoam`. Тип задачи ламинарный. Далее копируем учебный пример `blockedChannel` в свою рабочую директорию. Его возьмем за основу. Переименуйте его в `Re_41`, удалите системные файлы `Allclean`, `Allrun`.

2 ВЫЧИСЛЕНИЕ СКОРОСТИ

Воспользуемся формулой для вычисления числа Рейнольдса:

$$Re = \frac{d|U|}{\nu} \quad (1)$$

Вязкость воды при температуре 293.2 равна $1.006 \cdot 10^{-6} \text{ м}^2/\text{с}$. Диаметр цилиндра по условию задачи равен 0.01 м. Можем вычислить скорость:

$$U = \frac{Re \cdot \nu}{d} \quad (2)$$

Для числа Рейнольдса 41 скорость будет равна:

$$U = \frac{41 \cdot 1.006 \cdot 10^{-6}}{0.01} = 0.0041246 \text{ м/с} \quad (3)$$

Для числа Рейнольдса 140 скорость будет равна:

$$U = \frac{140 \cdot 1.006 \cdot 10^{-6}}{0.01} = 0.014084 \text{ м/с} \quad (4)$$

3 НАСТРОЙКА ФАЙЛОВ РЕШЕНИЯ

Заходим в директорию 0 для настройки граничных условий и начальных полей. Нам нужны только файлы скорости и давления, поэтому третий файл удаляем. Начальное поле распределения давления оставляем без изменений: `internalField uniform 0`. В нашей задаче будет 5 граничных условий: вход (`inlet`), выход (`outlet`), цилиндр (`cylinder`), верхняя и нижняя стенки (`wall`), передняя и задняя стенки (`frontAndBack`). В файле `p` указываем тип `empty` для грани `frontAndBack`, так как нам не нужно выполнять расчет относительно этих граней. На выходе (`outlet`) устанавливаем давление 0, остальным граням задаем нулевой градиент. Значение нуля означает, что градиент соответствующей величины (в данном случае давления) равен нулю, что показывает, что фактическое значение является постоянным.

Листинг 1 – Настройка файла давления

```
dimensions      [0 2 -2 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    inlet
    {
        type      zeroGradient;
    }
    outlet
    {
        type      fixedValue;
        value      uniform 0;
    }
    wall
    {
        type      zeroGradient;
    }
    cylinder
    {
        type      zeroGradient;
    }
    frontAndBack
    {
        type      empty;
    }
}
```

В файле скорости указываем начальное поле распределения равное нулю: `internalField uniform (0 0 0)`. По аналогии указываем тип `empty` грани `frontAndBack`, `inlet` задаем тип фиксированного значения и указываем вычисленную скорость как `uniform (0.0041246 0 0)` для числа Рейнольдса 41, на выходе нулевой градиент, а стенке и цилиндру задаем тип `fixedValue`, что означает «без прилипания» (адиабатическая стенка). Также тип этого условия можно записать как `noSlip` и без значения `value`. Это будет означать то же самое.

Листинг 2 – Настройка файла скорости

```
dimensions      [0 1 -1 0 0 0 0];

internalField    uniform (0 0 0);

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform (0.0041246 0 0);
    }
    outlet
    {
        type      zeroGradient;
    }
    wall
    {
        type      fixedValue;
        value      uniform (0 0 0);
    }
    cylinder
    {
        type      fixedValue;
        value      uniform (0 0 0);
    }

    frontAndBack
    {
        type      empty;
    }
}
```

Переходим в директорию constant. Тут оставляем только файл transportProperties. Настраиваем в нем вязкость воды. Значение берем для температуры 293.2 К.

Листинг 3 – Настройка вязкости

```
transportModel  Newtonian;

nu              [0 2 -1 0 0 0 0] 1.006e-6;
```

Создаем в этой же директории новый файл turbulenceProperties, в котором укажем тип течения ламинарный.

Листинг 4 – Настройка файла turbulenceProperties

```
/*-----*- C++ -*-----*\
=====
\\      / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\      / O peration  | Website:  https://openfoam.org
\\      / A nd        | Version:  9
\\      / M anipulation|
\*-----*/
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "constant";
    object      RASProperties;
}
// *****
simulationType laminar;
// *****
```

Переходим в каталог system. Удаляем файл generateAlphas, для нашей задачи он не нужен. Файл fvSchemes оставляем без изменений, в нем указаны схемы, которые будут использоваться при расчете.

В файле fvSolution необходимо сделать некоторые изменения. Изменим допуск при вычислении давления.

Также установим параметр nNonOrthogonalCorrectors = 1. Он определяет повторяющиеся решения уравнения давления, используемые для обновления явной неортогональной коррекции.

Листинг 5 – Настройка файла решения fvSolution

```
solvers
{
    p
    {
        solver          GAMG;
        smoother         DICGaussSeidel;
        tolerance        1e-6;
        relTol           0.01;
    }

    pFinal
    {
        $p;
        relTol           0;
    }

    "(U|tracer)"
    {
        solver          PBiCGStab;
        preconditioner   DILU;
        tolerance        1e-05;
        relTol           0.1;
    }

    "(U|tracer)Final"
    {
        $U;
        relTol           0;
    }
}

PIMPLE
{
    nNonOrthogonalCorrectors 1;
    nCorrectors              2;
}
```

Осталось настроить файл controlDict. В нем задается используемый решатель, в данном случае pimpleFoam. Укажем время завершения расчета 100 с, зададим временной шаг и интервал записи решений. Будем записывать результат каждые две секунды. Эти значения можно менять. Поле startFrom latestTime говорит решателю продолжать решение при запуске.

Листинг 6 – Настройка файла controlDict

```
application      pimpleFoam;

startFrom        latestTime;

startTime        0;

stopAt           endTime;

endTime          100;

deltaT           0.1;

writeControl      adjustableRunTime;

writeInterval     2;

purgeWrite        0;

writeFormat       ascii;

writePrecision    6;

writeCompression off;

timeFormat        general;

timePrecision     6;

runTimeModifiable yes;

adjustTimeStep    yes;

maxCo             5;

functions
{
    #includeFunc scalarTransport(tracer)
}
```

4 ПОСТРОЕНИЕ СЕТКИ

Сетку будем строить с помощью утилиты blockMesh. Для этого необходимо отредактировать файл blockMeshDict, расположенный в директории system.

По задаче диаметр окружности равен 0.01 м. В начале файла зададим перевод единиц измерения: convertToMeters 0.01. Теперь можем указывать диаметр равным 1. Высоту примем 10, всю ширину 60 (левая крайняя точка будет иметь координату по оси X -10, а крайняя правая точка 50). Ширина блока около цилиндра – 10. Для вычисления координат точек квадрата, который будет вписан в окружность, воспользуемся теоремой Пифагора.

Открываем файл blockMeshDict и последовательно создаем 23 точки.

Листинг 7 – Создание точек

```
Vertices
(
    (-10 -5 0.5) // 0
    (-5 -5 0.5) // 1
    (-5 5 0.5) // 2
    (-10 5 0.5) // 3
    (-10 -5 -0.5) // 4
    (-5 -5 -0.5) // 5
    (-5 5 -0.5) // 6
    (-10 5 -0.5) // 7
    (5 -5 0.5) // 8
    (50 -5 0.5) // 9
    (50 5 0.5) // 10
    (5 5 0.5) // 11
    (5 -5 -0.5) // 12
    (50 -5 -0.5) // 13
    (50 5 -0.5) // 14
    (5 5 -0.5) // 15
    (-0.353553391 -0.353553391 0.5) // 16
    (0.353553391 -0.353553391 0.5) // 17
    (0.353553391 0.353553391 0.5) // 18
    (-0.353553391 0.353553391 0.5) // 19
    (-0.353553391 -0.353553391 -0.5) // 20
    (0.353553391 -0.353553391 -0.5) // 21
    (0.353553391 0.353553391 -0.5) // 22
    (-0.353553391 0.353553391 -0.5) // 23
);
```

Следующим шагом перечисляем последовательно вершины, которые образуют каждый блок. В скобках указываем количество ячеек по каждому

направлению. По оси Y будет 30 ячеек, по оси X тоже 30, исключение правый блок, там 200, по оси Z одна ячейка, чтобы обеспечить 2D решение поставленной задачи.

Листинг 8 – Формирование блоков

```
blocks
(
  // left block
  hex ( 0 1 5 4 3 2 6 7) (30 1 30) simpleGrading (1 1 1)
  // cylinder blocks
  hex (1 8 12 5 16 17 21 20) (30 1 30) simpleGrading (1 1 1) // bottom
  hex (1 16 20 5 2 19 23 6) (30 1 30) simpleGrading (1 1 1) // left
  hex (19 18 22 23 2 11 15 6) (30 1 30) simpleGrading (1 1 1) // top
  hex (17 8 12 21 18 11 15 22) (30 1 30) simpleGrading (1 1 1) // right
  // right block
  hex ( 8 9 13 12 11 10 14 15) (200 1 30) simpleGrading (1 1 1)
);
```

Далее переходим к обозначению дуг, которые будут образовывать окружность цилиндра. Перечисляем две точки и смещение по оси, не забываем, что центр окружности совпадает с центром координат, поэтому смещение будет на величину радиуса – 0.5.

Листинг 9 – Создание дуг, образующие цилиндр

```
edges
(
  arc 16 17 (0 -0.5 0.5)
  arc 21 20 (0 -0.5 -0.5)
  arc 19 16 (-0.5 0 0.5)
  arc 23 20 (-0.5 0 -0.5)
  arc 19 18 (0 0.5 0.5)
  arc 23 22 (0 0.5 -0.5)
  arc 18 17 (0.5 0 0.5)
  arc 22 21 (0.5 0 -0.5)
);
```

Полностью удаляем блок с defaultPatch, эти грани будут обозначены в граничных условиях.

Далее обозначаем границы. В данной задаче имеем грань на входе, на выходе, стенки (верхняя и нижняя), цилиндр, передняя и задняя грани (не будут принимать участие в расчете). Обозначим последние empty. Цилиндр и стенки имеют тип wall, вход и выход – patch.

Листинг 10 – Граничные условия

```
boundary
(
  inlet
  {
    type patch;
    faces
    (
      ( 0 3 7 4)
    );
  }
  outlet
  {
    type patch;
    faces
    (
      (9 13 14 10)
    );
  }
  wall
  {
    type wall;
    faces
    (
      (3 2 6 7)
      (2 11 15 6)
      (11 10 14 15)
      (0 1 5 4)
      (1 8 12 5)
      (8 9 13 12)
    );
  }
  cylinder
  {
    type wall;
    faces
    (
      (16 17 21 20)
      (17 18 22 21)
      (19 18 22 23)
      (16 19 23 20)
    );
  }
  frontAndBack
  {
    type empty;
    faces
    (
      ( 0 1 2 3)
      ( 1 16 19 2)
      (19 18 11 2)
      (17 8 11 18)
      ( 1 8 17 16)
      (8 9 10 11)
      ( 4 7 6 5)
      ( 5 6 23 20)
      (23 6 15 22)
      (21 22 15 12)
      (5 20 21 12)
      (12 15 14 13)
    );
  }
);
```

Файл готов, можно проверять. Вводим в терминале из корневой папки задачи (Re_41) команду построения сетки – `blockMesh`. В директории `constant` будет создана папка `polyMesh`. Посмотрим полученную сетку с помощью `paraView`. Вводим в терминале `paraFoam`, выбираем отображение – `Surface With Edge`. Должен получиться результат, как показан на рисунке 2.

Также можно просто посмотреть полученные блоки, для этого запустить `paraView` с соответствующим флагом: `paraFoam -block` (Рисунок 3).

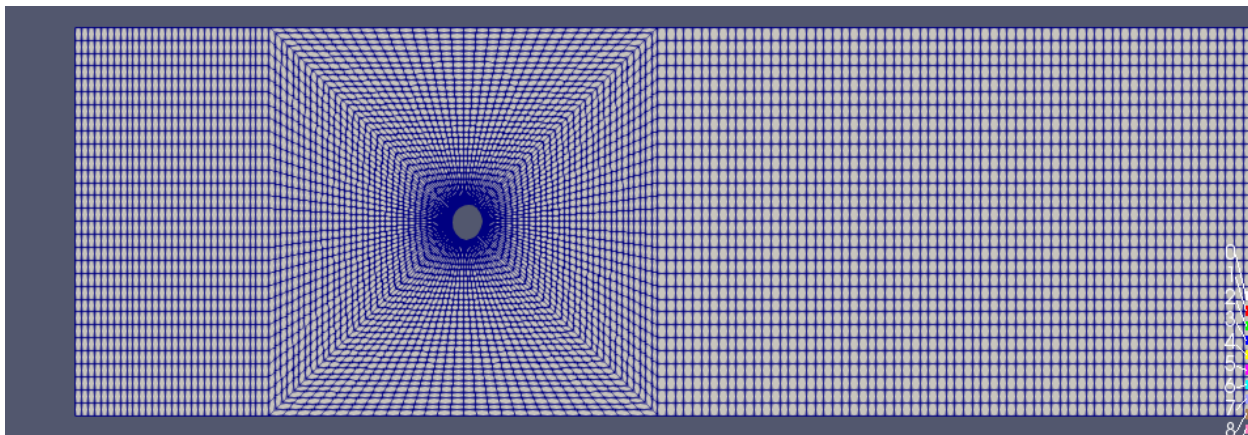


Рисунок 2 – Сетка в paraView

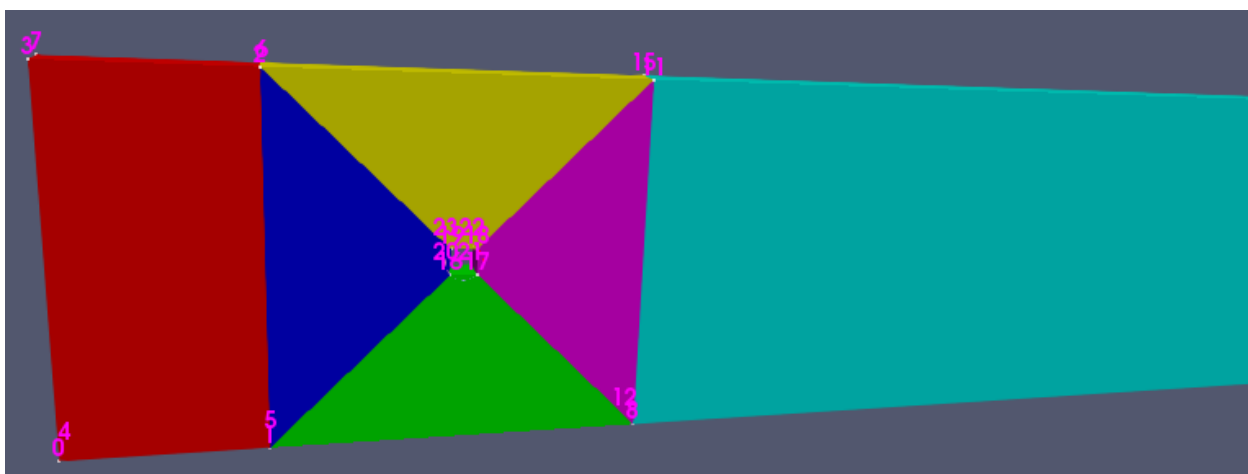


Рисунок 3 – Блоки сетки в paraView

5 РАСЧЕТ 1

Из корневой папки задачи запускаем в терминале решатель с помощью команды `pimpleFoam`. После завершения расчета открываем `paraView` с помощью команды `paraFoam`. Нажимаем кнопку `Apply`, выбираем отображение скорости по оси X. Запускаем анимацию. На момент времени 100 с должен получиться результат, показанный на рисунке 4.

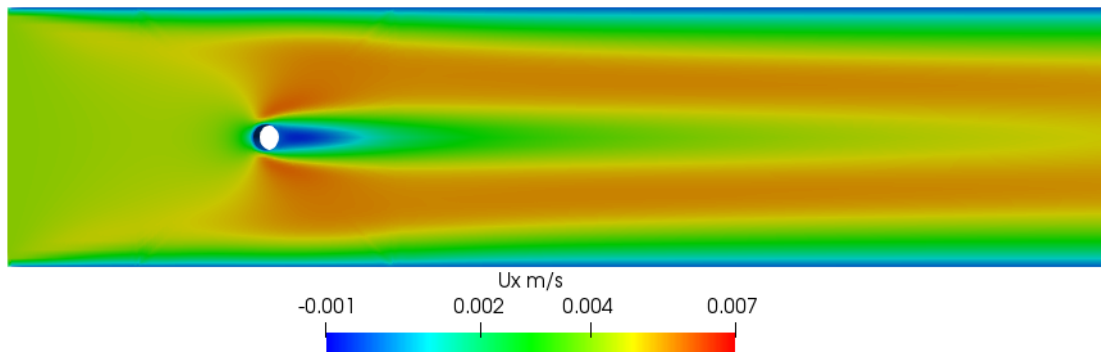


Рисунок 4 – Скорость потока при числе $Re = 41$

Изменить формат цветов отображения можно на стандартный. Для этого нажмите на значок  на панели свойств и выберите нужный (rainbow). Если его нет, нажмите на шестеренку для открытия скрытых вариантов. Также обязательно нажмите на кнопку `Rescale to data range over all timesteps`, чтобы легенда подстроилась под значения для текущего времени.

Теперь будем строить линии тока. Для этого выбираем на верхней панели инструмент `Stream Tracer` . Также его можно найти в меню `Filter`. Установим `Seed Type` в значение `Point Source`, изменим радиус сферы на 0.05, чтобы охватить всю высоту, и выключим ее отображение (Рисунок 5). Нажмите кнопку `Apply`.

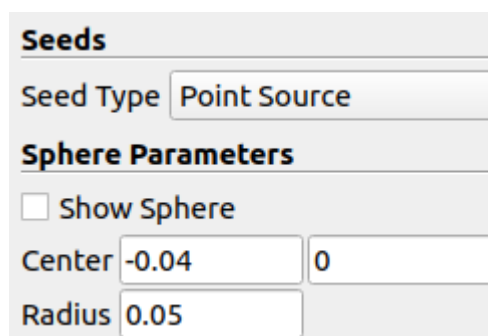


Рисунок 5 – Настройка `Stream Tracer`

Количество линий регулируется свойством Number of Points. Для того, чтобы отобразить геометрию вместе с траекторией, необходимо включить ее отображение (нажать на значок глаза в Pipeine Browser), затем установить прозрачность (свойство opacity) на 0.1, например. Также можно отобразить только половину геометрии, для этого воспользуемся инструментом Clip, выберем необходимую площадь сечения, нажмем Apply, установим прозрачность. Результат должен быть таким, как на рисунке 6.

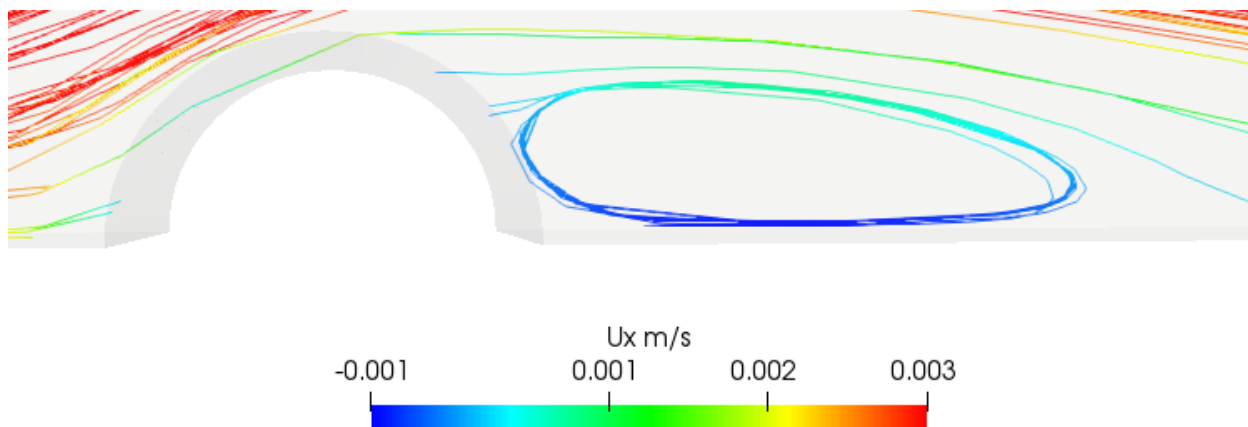


Рисунок 6 – Траектории потока при $Re = 41$

Результаты моделирования такого потока из SOLIDWORKS FLOW SIMULATION 2018 (validations examples) вместе с фотографией такого потока изображены на рисунке 7.

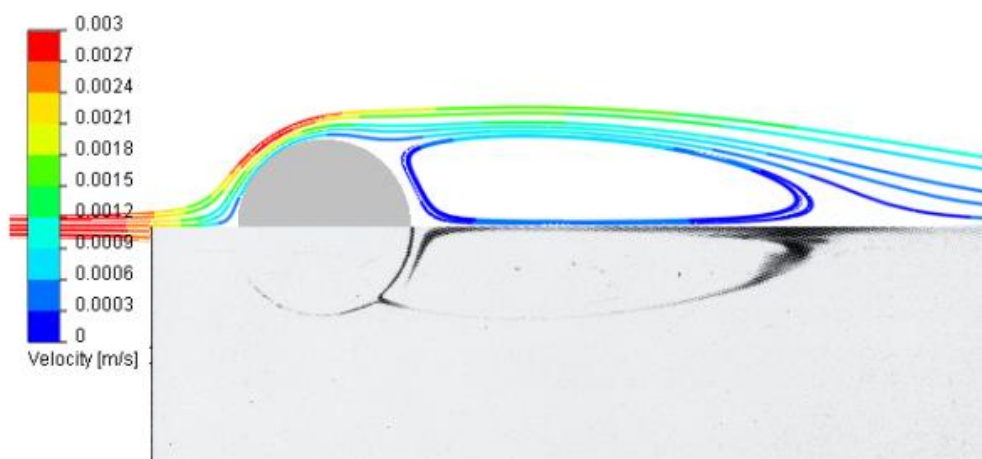


Рисунок 7 – Траектории потока при $Re = 41$ SolidWorks и фотография потока

Как видно OpenFOAM справился с этим расчетом, погрешность есть, но она не так критична.

6 РАСЧЕТ 2

Теперь выполним расчет с числом Рейнольдса = 140. Создадим новую папку, назовем ее Re_140. Из предыдущего решения копируем папки 0, constant и system. Изменяем значение скорости согласно вычислениям ранее. Больше никаких изменений делать не нужно. Сетка уже построена, поэтому запускаем расчет с помощью команды `pimpleFoam`. На 72 секунде должен получиться следующий результат (Рисунок 8).

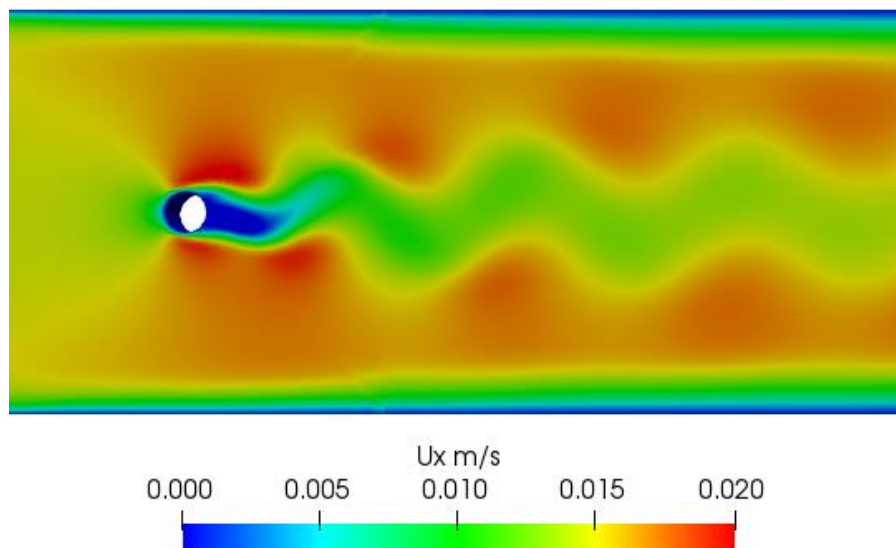


Рисунок 8 – Скорость при $Re = 140$

Если будем сравнивать с результатами SOLIDWORKS FLOW SIMULATION 2018 (validations examples), изображенном на рисунке 9, то заметим, что решатель `pimpleFoam` справляется с данным расчетом, результаты отличаются незначительно.

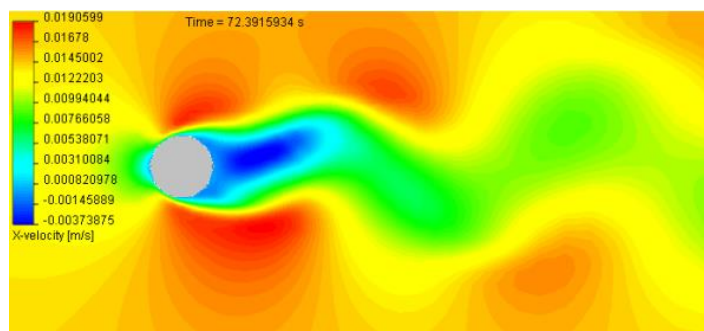


Рисунок 9 – Скорость при $Re = 140$ SolidWorks

ЗАКЛЮЧЕНИЕ

В ходе данной работы был выполнен расчет задачи обтекания цилиндра при заданных числах Рейнольдса с учетом вычисленной скорости для каждого из вариантов. Сравнение с результатами, полученными из SOLIDWORKS FLOW SIMULATION 2018 (validations examples), показало, что pimpleFoam из группы решателей, предназначенных для несжимаемого потока, справляется с решением такого рода задач с незначительной погрешностью.

Вопросы для самоконтроля:

1. Что происходит с потоком при высоких числах Рейнольдса?
2. Как можно построить линии тока в ParaView?
3. Что означает нулевой градиент в граничных условиях?
4. Чем отличается ламинарное течение от турбулентного?
5. Что означает тип noSlip в граничных условиях? Как можно записать его иначе?